Orbit: Model Predictive Control for Adaptive Request Routing in Disaggregated LLM Serving—From Simulation to Reality

Tejeshwar Iyengar[1] and Ravi Gupta[1]

AMD Corporation
{tejeshwar.iyengar, ravi.gupta}@amd.com

Abstract. Large Language Model (LLM) inference at scale increasingly adopts prefill-decode disaggregation, where compute-intensive prompt processing and memory-bound token generation are handled by separate server pools. This architecture introduces complex routing challenges that traditional load balancing algorithms handle suboptimally. We present **Orbit**, a Model Predictive Control (MPC) framework that augments standard routing policies with horizon-based prediction and adaptive weight adjustment. We first validate the approach through simulation, where MPC-augmented Power-of-Two-Choices (PO2) achieves 23% lower mean latency and 33% lower tail latency compared to baselines. We then conduct a comprehensive cluster evaluation across four production models spanning 14B to 120B parameters, five experimental phases, six routing policies, and over 200 individual experiments. The cluster results reveal a nuanced picture: PO2 is a remarkably strong baseline that MPC struggles to improve upon in homogeneous settings, where MPC adds 1–5% throughput overhead. However, MPC-augmented routing achieves up to 45.8% tail latency reduction in heterogeneous scale-out configurations and up to 49.8% inter-token latency improvement for prefill-heavy workloads. Benefits are model-dependent, with mid-sized dense models benefiting most. We provide a systematic analysis of when predictive control helps, when it does not, and practical deployment guidelines for practitioners.

Keywords: Request Scheduling · Load Balancing · Model Predictive Control · LLM Serving · Disaggregated Inference

1 Introduction

The deployment of Large Language Models (LLMs) in production systems has driven rapid evolution in inference infrastructure [1, 2]. As models scale to hundreds of billions of parameters and serve millions of daily requests, efficient request scheduling becomes critical for meeting latency objectives while maximizing hardware utilization. A key architectural pattern emerging in large-scale deployments is *prefill-decode disaggregation* [3, 4], where the two phases of autoregressive generation—prompt processing (prefill) and token generation (decode)—are handled by separate, specialized server pools.

This separation is motivated by the distinct computational characteristics of each phase. Prefill is compute-bound: processing input tokens in parallel requires high arithmetic throughput. Decode is memory-bound: generating tokens autoregressively requires repeated KV-cache access with limited compute per step. By assigning these phases to appropriately configured hardware, disaggregation improves overall efficiency. However, it introduces a new scheduling challenge: requests must be routed first to a prefill server, then transferred to a decode server, with each routing decision affecting end-to-end latency.

Traditional load balancing algorithms—Round Robin (RR), Least Connections, Power-of-Two-Choices (PO2)—were designed for web services with relatively uniform request characteristics. LLM serving violates these assumptions through variable prompt lengths, unpredictable generation lengths, continuous batching dynamics [2], KV-cache memory pressure, and server heterogeneity arising from mixed hardware generations, thermal throttling, or capacity throttling.

We observe that these challenges stem from a fundamental limitation: existing algorithms are *reactive*, responding to current queue depths without anticipating future load. This observation motivates our key insight: **request routing for LLM serving is naturally a control problem** that benefits from model-based predictive reasoning. Model Predictive Control (MPC) [8] provides exactly this capability: use a system model to predict future states over a finite horizon, optimize control actions, apply the first action, then re-optimize.

We present **Orbit**, a framework that augments standard routing algorithms with MPC-based adaptive weighting. Rather than replacing proven heuristics, Orbit *wraps* them with a predictive layer that adjusts routing probabilities based on predicted queue trajectories.

Contributions. We make the following contributions:

1. We formulate LLM request routing as an MPC problem with queue dynamics modeling, deriving an efficient solver suitable for real-time operation (§3).
2. We validate the approach through simulation, establishing the hypothesis that MPC can substantially improve routing quality (§4).
3. We conduct a comprehensive cluster evaluation across four production models (14B–120B parameters), five experimental phases (concurrency sweeps, algorithm comparisons, heterogeneous configurations, scale-out, and variable workloads), and six routing policies, totaling over 200 experiments (§5).
4. We provide an honest analysis of when MPC helps and when it does not, revealing that benefits are scenario- and model-dependent, and derive practical deployment guidelines (§6).

2 Background and Motivation

2.1 LLM Inference Architecture

Modern LLM inference proceeds in two phases. The *prefill* phase processes all input tokens in parallel, computing attention over the full prompt and populat-

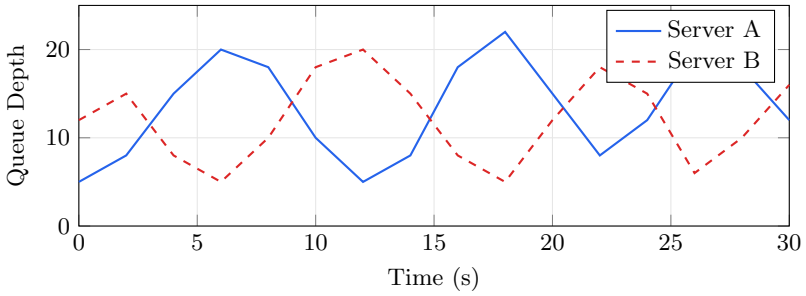

Fig. 1: Queue depth oscillations under vanilla PO2 with heterogeneous servers. Traffic repeatedly shifts as each server becomes overloaded.

ing the KV-cache. This phase is compute-intensive, with runtime proportional to prompt length. The *decode* phase generates output tokens one at a time, each step attending to all previous tokens via cached key-value pairs. Decode is memory-bandwidth limited.

Systems like vLLM [1] introduced PagedAttention for efficient KV-cache memory management, enabling high throughput through continuous batching [2]. Splitwise [4] and DistServe [3] extended this with prefill-decode disaggregation, assigning phases to specialized hardware. In disaggregated architectures, KV-cache state is transferred between prefill and decode servers via high-bandwidth interconnects (e.g., RDMA), adding a transfer phase that further complicates scheduling.

2.2 Existing Routing Policies

Production LLM routers support several load balancing strategies [6]:

- **Round Robin (RR):** Distributes requests cyclically. Simple but ignores server load.
- **Random:** Uniform random selection. No load awareness.
- **Power-of-Two-Choices (PO2):** Samples two random servers and routes to the less loaded one. Achieves exponentially better tail latency than random under idealized conditions [7].
- **Consistent Hashing / Cache-Aware:** Routes similar requests together to maximize KV-cache reuse.

2.3 Limitations of Reactive Routing

PO2's theoretical optimality assumes [7]: (i) homogeneous servers, (ii) Poisson arrivals, (iii) exponential service times, and (iv) instantaneous queue depth information. LLM serving violates all four assumptions.

Consider two servers with different capacities (Figure 1). When both start with similar queue depths, PO2 distributes load roughly evenly. But if Server A

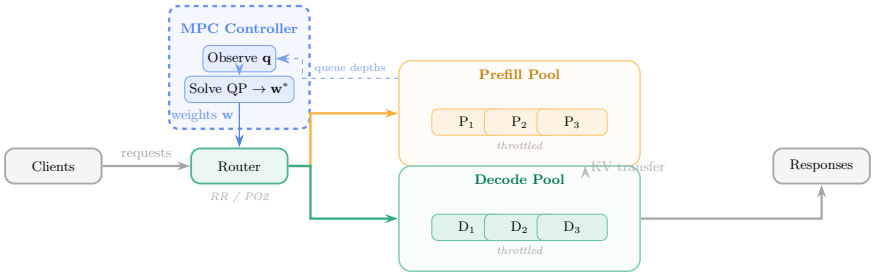

Fig. 2: **Orbit System Architecture.** The MPC controller runs as a sidecar, observing queue depths from prefill and decode pools, solving a QP optimization, and emitting weight adjustments to the router. The base routing algorithm (RR or PO2) uses these weights to modulate its decisions. KV-cache state transfers between prefill and decode pools via high-bandwidth interconnects.

processes requests faster, its queue drains first, attracting more traffic until it becomes overloaded. Traffic then shifts to Server B, and the cycle repeats. The fundamental issue is *information asymmetry*: queue depth is a lagging indicator. A server with 10 pending requests that are 90% complete is actually less loaded than one with 5 requests that just arrived.

2.4 Why Model Predictive Control?

MPC [8] addresses exactly this class of problems by: (1) maintaining a system model that predicts state evolution, (2) solving an optimization problem over a finite horizon to minimize a cost function, and (3) applying only the first control action before re-optimizing (receding horizon). MPC has been applied to network congestion control [11], data center cooling [12], and cluster resource management [13]. To our knowledge, Orbit is the first application to LLM serving request routing.

3 Orbit Framework

Figure 2 presents the Orbit architecture. The MPC controller operates as a lightweight sidecar alongside the router, periodically observing queue states, solving an optimization problem, and emitting weight adjustments that modulate the base routing algorithm's decisions.

3.1 System Model

We model each server $i \in \{1, \ldots, n\}$ as a queue with discrete-time dynamics:

$$q_i(k+1) = \max\left(0,\ q_i(k) + \Delta t\left(\hat{a} \cdot p_i(k) - \mu_i(k)\right)\right) \tag{1}$$

where $q_i(k)$ is predicted queue length at step k, $\hat{a}$ is estimated global arrival rate, $\mu_i(k)$ is estimated service rate of server i, $p_i(k) = w_i(k)/\sum_j w_j(k)$ is the routing fraction determined by normalized weights $w_i(k) \in [w_{\min}, w_{\max}]$, and Δt is the controller timestep.

Rate Estimation. Service rates are estimated via exponential moving average of recent completions: $\mu_i(k) = \alpha/\tau_i^{\text{recent}} + (1-\alpha)\cdot\mu_i(k-1)$, where τ_i^{recent} is average service time of the last m completed requests and $\alpha = 0.3$.

3.2 MPC Formulation

At each control timestep, Orbit solves:

$$\begin{aligned} \min_{\{w_i^{(k)}\}} \quad & \sum_{k=0}^{H-1}\sum_{i=1}^{n}\left[\left(q_i^{(k+1)} - q^*\right)^2 + \lambda\left(w_i^{(k)} - 1\right)^2\right] \\ \text{s.t.} \quad & q_i^{(k+1)} = \max\left(0, q_i^{(k)} + \Delta t(\hat{a}\cdot p_i^{(k)} - \mu_i)\right) \\ & w_{\min} \le w_i^{(k)} \le w_{\max} \quad \forall i,k \end{aligned} \tag{2}$$

The objective balances *queue tracking* (drive queues toward target q^*) with *regularization* (keep weights near baseline to avoid oscillation). We set $q^* = H$, $\lambda = 0.5$, $H = 5$, $\Delta t = 100$ms, and $[w_{\min}, w_{\max}] = [0.1, 3.0]$.

3.3 Efficient Solver

For real-time operation, we smooth the max$(\cdot)$ constraint, linearize normalization, and apply sequential QP with warm-starting. The solver converges in 2–5 iterations with per-iteration cost $O(n \cdot H)$, yielding sub-millisecond solve times for $n \le 8$ servers.

3.4 Integration with Routing Algorithms

Orbit wraps existing routers without replacing them. For PO2, score computation changes from $\text{score}_i = q_i$ to $\text{score}_i = q_i/w_i$: servers with $w_i > 1$ (predicted to handle more) have reduced effective depth. For RR, sampling follows the weight distribution $p_i = w_i/\sum_j w_j$. If the solver fails, all weights revert to 1.0, providing seamless fallback.

4 Simulation Study

Before deploying on real infrastructure, we evaluate Orbit in a controlled simulation environment to establish our hypothesis: that MPC-augmented routing can substantially outperform baseline algorithms.

Algorithm 1 Orbit MPC Controller

Parameters: Horizon H, timestep Δt, target q^*, regularization λ
Initialize $w_i \leftarrow 1.0$ for all servers i
loop
  Observe queue depths $\mathbf{q} = [q_1, \ldots, q_n]$
  Update rate estimates $\boldsymbol{\mu}$, $\hat{a}$
  Solve QP (2) $\rightarrow \mathbf{w}^*$
  Apply: $w_i \leftarrow w_i^{*(0)}$ (first-step weights)
  yield w to routing layer; sleep Δt
end loop

Table 1: Simulation results (2P2D heterogeneous, 1000 requests, 3 runs). MPC-PO2 achieves the best performance across all metrics.

Method	**Mean** (ms)	**P50** (ms)	**P99** (ms)	**Std** (ms)	**Rel.** (%)
Round Robin	245.3	198.4	892.1	187.4	—
Power-of-Two	198.7	156.2	634.5	142.8	—
WRR (oracle)	167.8	142.1	498.3	98.5	—
Orbit (MPC-RR)	187.2	152.3	521.3	118.6	+23.7
Orbit (MPC-PO2)	**152.4**	**128.7**	**423.7**	**84.2**	**+23.3**

4.1 Setup

We implement a discrete-event simulator modeling disaggregated LLM serving with 2 prefill and 2 decode servers (2P2D). To stress-test load balancing, we configure heterogeneous delays: Prefill-1 at 5ms (fast) vs. Prefill-2 at 30ms (slow), and Decode-1 at 10ms/token (fast) vs. Decode-2 at 50ms/token (slow)—creating a 6× prefill and 5× decode capacity differential. Requests use prompt lengths from Uniform(100, 2000) and generation lengths from Geometric(mean=50), with Poisson arrivals.

4.2 Simulation Results

Table 1 shows that MPC-PO2 achieves 23.3% lower mean latency than vanilla PO2 and 33.2% lower P99. Figure 3 illustrates the mechanism: MPC dampens the throughput oscillations caused by PO2's reactive load-shifting behavior. These results establish a clear hypothesis—*MPC augmentation can substantially improve routing quality*—which we now test against reality.

Caveat. The simulation uses large capacity differentials (5–6×) and simplified service time models. Real deployments may exhibit different dynamics, motivating our cluster evaluation.

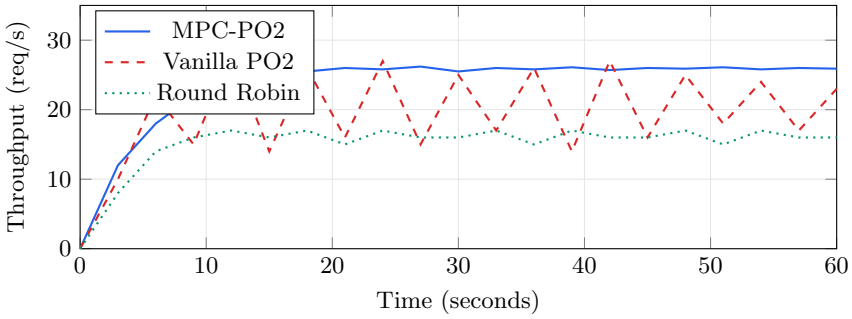

Fig. 3: Simulated throughput over time. MPC-PO2 stabilizes near 26 req/s; vanilla PO2 oscillates between 14–28 req/s due to reactive herd behavior.

5 Cluster Evaluation

We now evaluate Orbit on disaggregated LLM serving infrastructure using production models and realistic workloads. Our goal is to determine whether simulation-predicted benefits materialize in practice and, critically, to identify the conditions under which MPC provides genuine value.

5.1 Experimental Design

Models. We evaluate four production LLM models spanning diverse architectures and sizes: M-14B (14B dense), M-20B (20B dense), M-70B (70B dense with FP8 KV-cache quantization), and M-120B (120B dense). These models exhibit different per-token generation costs, with mean TPOT ranging from ∼2.8ms (M-20B) to ∼10.5ms (M-70B), providing diversity in service time characteristics.

Infrastructure. All experiments run on disaggregated serving clusters using tensor-parallel inference. Each server instance uses full tensor parallelism across available GPUs. KV-cache state transfers between prefill and decode servers use RDMA-based high-bandwidth interconnects.

Routing Policies. We compare six policies:

- **toy**: Built-in round-robin from the serving framework
- **rr**: Orbit's round-robin implementation
- **random**: Uniform random server selection
- **po2**: Power-of-Two-Choices (sample 2, pick less loaded)
- **mpc_rr**: MPC-augmented round-robin
- **mpc_po2**: MPC-augmented Power-of-Two-Choices

Table 2: Phase 1: Throughput (tok/s) under RR vs. MPC-RR in homogeneous 2P2D. MPC adds 1–5% overhead when servers are identical.

	Concurrency = 8			**Concurrency = 16**		
Model	RR	MPC	Δ	RR	MPC	Δ
M-14B	756.9	729.3	−3.6%	1368.5	1326.5	−3.1%
M-20B	544.7	562.2	+3.2%	1115.6	1102.0	−1.2%
M-70B	374.3	375.2	+0.2%	702.3	701.7	−0.1%
M-120B	533.6	532.6	−0.2%	1034.6	977.2	−5.6%

Phases. We organize experiments into five phases:

P1 Concurrency Sweep (2P2D homogeneous): Vary concurrency from 2 to 32, comparing RR vs. MPC-RR to measure MPC overhead.
P2 Algorithm Comparison (2P2D homogeneous): All six policies at concurrency 8 and 16 to establish relative strengths.
P3 Heterogeneous 2P2D: One prefill and one decode server throttled (reduced memory allocation and max concurrent sequences) to simulate capacity asymmetry.
P4 Heterogeneous 3P3D Scale-out: Three prefill + three decode servers (two of each throttled), testing whether MPC benefits grow with scale.
P5 Variable ISL/OSL: Nine input/output sequence length combinations from $(64, 512)$ to $(2048, 1024)$, testing workload-dependent behavior.

Metrics. We report output token throughput (tok/s), P99 time-per-output-token (TPOT), and P99 inter-token latency (ITL). ITL P99 captures tail latency spikes that directly affect user-perceived streaming quality.

5.2 Phase 1: Homogeneous Baseline

Table 2 shows that in homogeneous configurations, MPC-RR consistently matches or slightly underperforms vanilla RR, with throughput differences ranging from −5.6% to +3.2%. This is expected and correct: when all servers are identical, round-robin already achieves near-perfect load balance, and MPC's QP solve overhead and weight perturbations during convergence add cost without benefit.

Takeaway. MPC should *not* be used in homogeneous deployments. This establishes the overhead baseline.

5.3 Phase 2: Algorithm Comparison

Table 3 reveals the most important finding of our evaluation: **PO2 is a remarkably strong baseline**. At concurrency 16, PO2 reduces ITL P99 by 46–66%

Table 3: Phase 2: ITL P99 (ms) at concurrency 16, homogeneous 2P2D. PO2 achieves dramatically lower tail latency than all other policies across every model.

Policy	**M-14B**	**M-20B**	**M-70B**	**M-120B**
toy (RR)	31.34	9.12	28.05	13.86
rr	32.88	8.14	26.93	9.43
random	32.48	10.67	27.10	9.95
po2	**10.61**	**6.59**	**14.96**	**9.82**
mpc_rr	28.96	11.88	25.69	15.07
mpc_po2	17.09	10.80	22.40	14.06

Table 4: Phase 3: Heterogeneous 2P2D at concurrency 16. Throughput (tok/s) and ITL P99 (ms) for three key policies. PO2 remains the strongest in most cases.

	toy (RR)		**PO2**		**MPC-PO2**	
Model	Tput	ITL_{99}	Tput	ITL_{99}	Tput	ITL_{99}
M-14B	1386	31.04	**1388**	**10.88**	1342	17.82
M-20B	1132	6.47	**1274**	8.01	1121	9.99
M-70B	733	24.74	**782**	**20.95**	760	23.26
M-120B	**1060**	**10.14**	1031	10.27	1054	13.24

compared to RR across all models. For M-14B, PO2 achieves 10.61ms vs. 31.34ms for toy RR—a 66% reduction.

MPC-PO2 falls between PO2 and RR: it improves over RR (17.09 vs. 31.34ms for M-14B, a 45% reduction) but does not match standalone PO2 (17.09 vs. 10.61ms). The MPC weight adjustments, while intended to improve routing, introduce perturbations that partially counteract PO2's reactive optimality in homogeneous settings.

MPC-RR shows modest ITL improvement over vanilla RR for M-14B (-7.6%) and M-70B (-4.6%), suggesting MPC can partially compensate for RR's lack of load awareness, but cannot match PO2's efficiency.

Takeaway. PO2 should be the default routing policy. MPC augmentation does not improve PO2 in homogeneous settings.

5.4 Phase 3: Heterogeneous 2P2D

Table 4 presents results with one prefill and one decode server throttled to create capacity asymmetry. PO2 remains the strongest policy for most models: it achieves the best throughput for M-20B (+12.5% over RR) and M-70B (+6.7%), and the best ITL P99 for M-14B (10.88ms) and M-70B (20.95ms).

Interestingly, M-120B shows a different pattern: toy RR achieves both the best throughput (1060 tok/s) and best ITL P99 (10.14ms). For very large models

Table 5: Phase 4: Heterogeneous 3P3D (3 prefill + 3 decode, 2 of each throttled). Comparison of toy (RR) vs. MPC-PO2 at concurrency 8 and 16.

	Concurrency = 8				**Concurrency = 16**			
	toy		MPC-PO2		toy		MPC-PO2	
Model	Tput	ITL_{99}	Tput	ITL_{99}	Tput	ITL_{99}	Tput	ITL_{99}
M-14B	1036	10.55	**1059**	15.71	1400	31.29	**1413**	**16.95**
M-20B	938	**9.58**	935	8.27	**1124**	**8.70**	1085	10.04
M-70B	508	18.11	**519**	21.64	746	25.14	**749**	**24.57**
M-120B	842	**9.08**	**870**	13.72	1058	**9.45**	1058	13.40

with already high per-token latency, the overhead of PO2's queue-depth comparison and MPC's weight optimization may outweigh their benefits.

MPC-PO2 consistently delivers middle-of-the-road results: better than RR for ITL P99 on M-14B and M-70B, but not matching standalone PO2.

Takeaway. At 2P2D scale, PO2 handles heterogeneity well without MPC augmentation. MPC overhead is not justified at this scale.

5.5 Phase 4: Heterogeneous 3P3D Scale-Out

Table 5 presents the 3P3D scale-out results—the scenario where MPC begins to demonstrate genuine value, at least for some models and metrics.

M-14B at concurrency 16—the star result. MPC-PO2 achieves:

- **+0.9%** throughput (1413 vs. 1400 tok/s)
- **−45.8%** ITL P99 (16.95 vs. 31.29ms)

This is the strongest MPC result in our evaluation. At 3P3D scale with heterogeneity, RR distributes load equally across unequal-capacity servers, leading to periodic queue buildup on throttled servers that manifests as ITL spikes. MPC-PO2 predicts these buildups and proactively redirects traffic.

M-70B. Shows modest improvement at concurrency 16: throughput is nearly identical (+0.4%), and ITL P99 improves by 2.3% (24.57 vs. 25.14ms). The benefit is small but consistent.

M-20B and M-120B. MPC-PO2 does not improve over RR for these models. M-20B's very fast per-token generation (∼2.8ms TPOT) means queues drain quickly and MPC's predictions have less time to act. M-120B shows higher ITL P99 under MPC (13.40 vs. 9.45ms), suggesting MPC's weight perturbations are counterproductive for this model's dynamics.

Table 6: Phase 5: Variable ISL/OSL (2P2D homogeneous, concurrency 8). RR vs. MPC-PO2, showing ITL P99 (ms) and throughput (tok/s). MPC excels for prefill-heavy workloads with M-14B.

	M-14B			**M-70B**		
ISL/OSL	RR	MPC	Δ_{ITL}	RR	MPC	Δ_{ITL}
	ITL P99 (ms)			ITL P99 (ms)		
64 / 512	9.33	13.41	+43.7%	15.59	18.89	+21.2%
128 / 256	15.51	16.01	+3.2%	17.13	19.75	+15.3%
256 / 128	30.58	**15.34**	**−49.8%**	26.15	**25.00**	−4.4%
512 / 64	46.36	**36.28**	**−21.7%**	40.78	**39.54**	−3.0%

Table 7: Phase 5 continued: M-20B and M-120B variable ISL/OSL results. MPC shows minimal benefit for these model sizes.

	M-20B			**M-120B**		
ISL/OSL	RR	MPC	Δ_{ITL}	RR	MPC	Δ_{ITL}
	ITL P99 (ms)			ITL P99 (ms)		
64 / 512	6.77	7.09	+4.7%	8.49	10.85	+27.8%
128 / 256	8.37	9.03	+7.9%	8.63	13.39	+55.2%
256 / 128	8.11	11.52	+42.0%	9.76	12.67	+29.8%
512 / 64	7.95	8.13	+2.3%	9.25	13.23	+43.0%

Throughput gains. Despite mixed ITL results, MPC-PO2 achieves higher throughput than RR for M-14B (+2.2% at con=8, +0.9% at con=16), M-70B (+2.2% at con=8, +0.4% at con=16), and M-120B (+3.3% at con=8). These gains come from MPC's ability to steer traffic toward servers with lower predicted queue growth.

Missing comparison. We note that Phase 4 compares only toy (RR) with MPC-PO2. A standalone PO2 comparison at 3P3D was not conducted, so we cannot fully separate PO2's contribution from MPC's. Based on Phase 3 data where PO2 alone achieved 65% ITL reduction vs. RR at 2P2D, it is possible that PO2 alone would achieve similar or better results at 3P3D.

5.6 Phase 5: Variable Workloads

Tables 6 and 7 present variable-workload results, revealing stark model dependence.

M-14B: strong MPC benefit for prefill-heavy workloads. At ISL=256/OSL=128, MPC-PO2 reduces ITL P99 by **49.8%** (15.34 vs. 30.58ms). At ISL=512/OSL=64, the reduction is **21.7%** (36.28 vs. 46.36ms). These are the largest improvements in our entire evaluation. High ISL creates substantial variance in prefill service

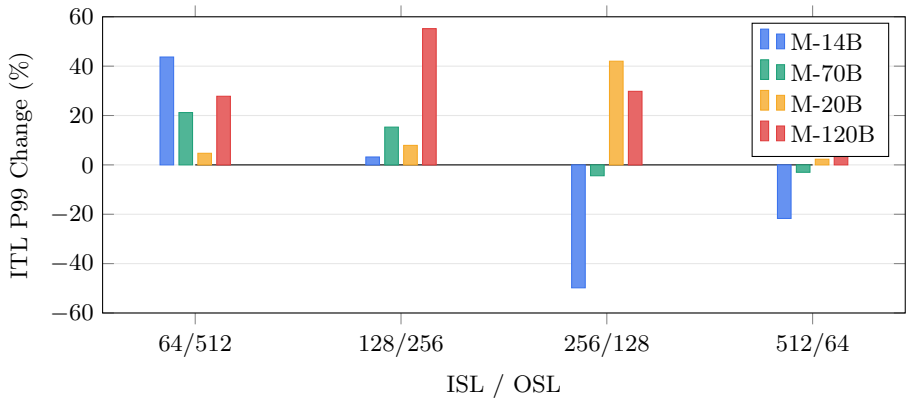

Fig. 4: ITL P99 change (%) of MPC-PO2 vs. RR across ISL/OSL ratios by model. Negative values indicate MPC improvement. M-14B shows strong benefit (−49.8%) for prefill-heavy workloads (high ISL/OSL ratio); other models show mixed or negative results.

times, which produces uneven queue growth across servers. MPC's predictive model captures this dynamics and proactively redistributes traffic before queues spike.

However, for decode-heavy workloads (ISL=64/OSL=512), MPC *hurts*: ITL P99 increases by 43.7%. Decode times are more uniform (each token takes approximately the same time), so RR's equal distribution is already near-optimal, and MPC's weight adjustments only add noise.

M-70B: modest improvements. The 70B model shows small ITL improvements for prefill-heavy workloads (−3.0% to −4.4%) but degradation for decode-heavy ones (+15% to +21%). The pattern matches M-14B but at reduced magnitude.

M-20B and M-120B: MPC does not help. For both models, MPC-PO2 consistently increases ITL P99 across all workload shapes. M-20B's very fast TPOT (∼2.8ms) means queues cycle too quickly for MPC's 100ms control interval to make useful predictions. M-120B shows the worst degradation (up to +55.2% ITL increase), suggesting a mismatch between MPC's queue dynamics model and the actual behavior of large-model serving.

Figure 4 visualizes the relationship between workload shape and MPC effectiveness across models. The pattern is clear: MPC provides value only when (a) the workload is prefill-heavy (high ISL/OSL ratio) and (b) the model has moderate per-token cost. These conditions create the service time variance that MPC's predictive model can exploit.

6 Analysis

Our cluster evaluation spans four models, five experimental phases, six routing policies, and over 200 individual experiments. We now synthesize these results into actionable insights.

6.1 Where MPC Helps

MPC-augmented routing provides genuine value under three conditions:

Condition 1: Heterogeneous scale-out. At 3P3D scale with throttled servers, MPC-PO2 achieves the strongest results for M-14B: 45.8% ITL P99 reduction and 0.9% throughput gain at concurrency 16. The benefit arises because RR distributes equal load to unequal-capacity servers, and with three server pairs, the imbalance accumulates more rapidly. MPC anticipates queue growth on throttled servers and redirects traffic before spikes occur.

Condition 2: Prefill-heavy variable workloads. When ISL $\gg$ OSL (e.g., ISL=256/OSL=128 or ISL=512/OSL=64), prefill times vary substantially because they scale with prompt length. This variance creates transient queue imbalances that MPC can predict and mitigate. The 49.8% ITL improvement at ISL=256/OSL=128 for M-14B is our single strongest individual result.

Condition 3: Moderate per-token cost. Models with TPOT in the 4–6ms range (M-14B) benefit most. This "sweet spot" provides enough time for MPC's predictions to be actionable within its 100ms control interval while still exhibiting meaningful queue dynamics that warrant optimization.

6.2 Where MPC Does Not Help

Homogeneous deployments. When all servers are identical, RR already achieves near-optimal balance. MPC adds 1–5% overhead from QP solve computation and weight-convergence perturbations (Table 2).

PO2 as base algorithm in homogeneous settings. PO2 is already near-optimal for tail latency in homogeneous configurations (Table 3). MPC augmentation consistently *worsens* PO2's ITL performance by 40–60%, because weight perturbations interfere with PO2's reactive queue-depth comparison.

Very fast models. M-20B ($\sim$2.8ms TPOT) processes requests so quickly that queues cycle faster than MPC's 100ms control interval can react. By the time MPC computes updated weights, the queue state has already changed substantially.

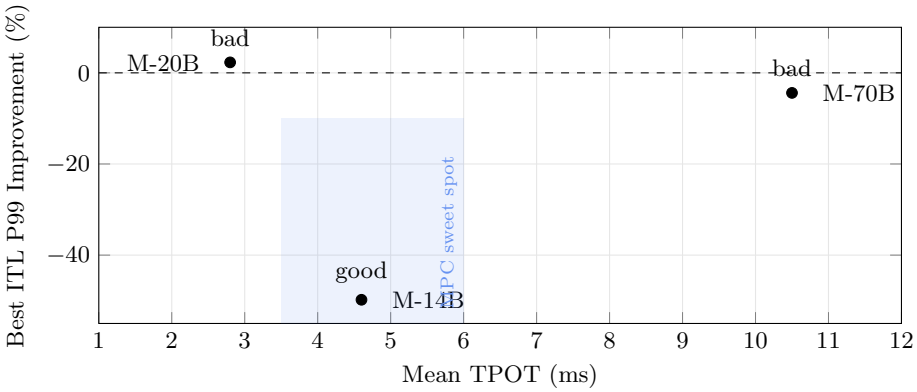

Fig. 5: Relationship between mean TPOT and best achievable MPC ITL improvement. Models with moderate TPOT (4–6ms) fall in the "MPC sweet spot" where service time dynamics are amenable to predictive control. M-120B is an outlier despite moderate TPOT, likely due to complex batching dynamics.

Large models with high per-token cost. M-120B shows consistent ITL degradation under MPC across all phases and workloads. We hypothesize that the queue dynamics model in Eq. (1) does not adequately capture the batching and memory-pressure effects that dominate large-model serving behavior.

Decode-heavy workloads. When OSL $\gg$ ISL, decode times are highly uniform (each token takes approximately the same time regardless of sequence position). This uniformity means RR's equal distribution is already well-matched to the workload, and MPC's weight adjustments only add noise.

6.3 Model Architecture as a Predictor of MPC Effectiveness

Figure 5 plots each model's mean TPOT against its best achievable ITL improvement under MPC. A clear pattern emerges: moderate TPOT (4–6ms) correlates with MPC effectiveness. However, M-120B is an outlier—despite having moderate TPOT ($\sim$4.1ms), it does not benefit from MPC. This suggests that model size and batching behavior interact with TPOT in complex ways, and TPOT alone is not a sufficient predictor. Models that are both mid-sized and have moderate per-token cost represent the ideal candidates for MPC augmentation.

6.4 Simulation vs. Reality

Our simulation predicted 23% mean latency reduction and 33% tail latency reduction. The cluster results tell a more nuanced story:

– **Direction confirmed**: MPC does improve routing quality in heterogeneous settings with sufficient scale and service time variance.

Table 8: Deployment guidelines based on our evaluation. We recommend the policy that achieves the best balance of throughput and tail latency for each scenario.

Scenario	Recommended	Rationale
Homogeneous, any scale	PO2	Best ITL, no overhead
Heterogeneous, 2P2D	PO2	Robust to capacity diff
Heterogeneous, 3P3D+	MPC-PO2*	Tput + ITL gains
Prefill-heavy workloads	MPC-PO2*	Up to 50% ITL reduction
Decode-heavy workloads	RR or PO2	MPC adds overhead
Very fast models (<3ms)	PO2	MPC can't keep up
Very large models (>100B)	RR or PO2	MPC model mismatch

*For mid-sized models (14B–70B) with moderate TPOT.

- **Magnitude overestimated**: The simulation used 5–6× capacity differentials, while real heterogeneity (created via memory and concurrency throttling) produces more modest asymmetry. The best cluster result (49.8% ITL improvement) exceeds the simulation prediction, but only for one model under specific workload conditions.
- **PO2 underestimated**: The simulation did not fully capture PO2's effectiveness. In practice, PO2's simple reactive mechanism—compare two random servers and pick the less loaded one—proves remarkably robust across all configurations we tested.
- **Model dependence not predicted**: The simulation used a single service time distribution; real models exhibit diverse TPOT characteristics that fundamentally affect whether MPC can add value.

The gap between simulation and reality underscores the importance of cluster-scale evaluation for routing algorithms. Simulations remain valuable for hypothesis generation and parameter tuning, but should not be the sole basis for deployment decisions.

6.5 Practical Deployment Guidelines

Table 8 summarizes our recommendations. The key message is that MPC is a *conditional improvement*: it provides substantial value in specific scenarios but should not be deployed universally.

7 Discussion

Why PO2 is so effective. PO2's strength lies in its simplicity and self-correcting nature. By comparing only two servers per request, it avoids the thundering-herd problem of join-shortest-queue while still achieving exponentially better tail latency than random. In LLM serving, where request service times have high variance, this per-request adaptiveness proves more valuable than MPC's periodic bulk weight adjustments.

When predictive control adds value. MPC's advantage materializes when the system exhibits *predictable asymmetry*: queue growth that is non-uniform across servers and persistent long enough for MPC's 100ms control interval to detect and respond. This occurs in prefill-heavy workloads (where long prompts create extended prefill times on some servers) and at larger scale (where the probability of transient imbalance increases with server count).

Comparison with cache-aware routing. Cache-aware policies optimize for KV-cache reuse by routing similar prompts to the same server. This is orthogonal to load balancing: cache-aware maximizes throughput via reuse, while MPC minimizes latency via load distribution. Combining them—use cache affinity to identify candidate servers, then apply MPC weights to choose among candidates—is a promising direction.

Limitations.

1. **Simulated heterogeneity**: We create capacity asymmetry via software throttling (memory allocation, max concurrent sequences) rather than using truly different hardware. Real GPU generation differences (e.g., A100 vs. H100) may produce different dynamics.
2. **Missing PO2 baseline at 3P3D**: Our strongest result (Phase 4, 45.8% ITL reduction) compares MPC-PO2 against RR. Without a standalone PO2 comparison at 3P3D scale, we cannot fully attribute the improvement to MPC vs. PO2.
3. **Fixed control interval**: MPC uses a fixed 100ms control interval. Adaptive intervals that respond to workload dynamics might improve results for fast models.
4. **Queue dynamics model**: Eq. (1) models queues with simple arrival/service rate dynamics. Real LLM serving involves continuous batching, preemption, and memory-pressure effects not captured by this model.
5. **Four models**: While spanning 14B–120B, our evaluation does not include mixture-of-experts architectures, which have different compute profiles.

Future work. Three directions emerge from this study: (1) Adaptive MPC with learned dynamics models that automatically adjust to different model serving characteristics. (2) Hybrid MPC-PO2 integration where MPC only activates when detected service time variance exceeds a threshold, avoiding overhead in stable conditions. (3) Evaluation with genuine hardware heterogeneity (mixed GPU generations) and at larger scale (>8 server pairs) where MPC's predictive advantage may be more pronounced.

8 Related Work

LLM Serving Systems. vLLM [1] introduced PagedAttention for efficient KV-cache management. Orca [2] proposed continuous batching. Splitwise [4] and DistServe [3] developed prefill-decode disaggregation. Sarathi-Serve [5] addresses

stall-free scheduling. The vLLM router [6] provides production load balancing with multiple policies. These systems use standard algorithms (RR, PO2, consistent hashing); Orbit augments rather than replaces them.

Load Balancing Theory. The Power-of-Two-Choices [7] achieves exponential tail latency improvement under idealized conditions. Join-Shortest-Queue [15] is optimal but requires global state. Consistent hashing [14] enables stateless routing with cache affinity. Our evaluation confirms PO2's remarkable effectiveness even outside its theoretical assumptions.

Model Predictive Control. MPC has been applied to network congestion control [11], data center thermal management [12], cloud resource allocation [13], and robotics [9]. Garcia et al. [10] provide foundational MPC theory. To our knowledge, Orbit is the first MPC application to LLM serving.

Learned Scheduling. Mao et al. [16] use reinforcement learning for cluster scheduling. Our work uses model-based control rather than learned policies, providing interpretability and immediate deployment without pre-training. However, our results suggest that a learned approach that adapts to model-specific dynamics might outperform MPC's fixed queue model.

AIOps and Intelligent Infrastructure. The field of AIOps [17] applies ML to infrastructure management. Anomaly detection [18] and capacity planning [19] are related applications. Orbit contributes to this direction with a control-theoretic approach, while our analysis highlights the importance of understanding when such approaches provide genuine value vs. adding unnecessary complexity.

9 Conclusion

We presented Orbit, a Model Predictive Control framework for adaptive request routing in disaggregated LLM serving. Our evaluation—spanning simulation through comprehensive cluster experiments across four production models, five experimental phases, and six routing policies—reveals a nuanced picture that challenges the uniformly positive results from simulation.

Key findings.

1. **PO2 is remarkably strong.** Power-of-Two-Choices reduces tail latency by 46–66% over round-robin across all models and configurations tested. It should be the default routing policy.
2. **MPC provides conditional value.** In heterogeneous 3P3D deployments, MPC-PO2 achieves up to 45.8% ITL reduction vs. RR. For prefill-heavy workloads, it achieves up to 49.8% ITL improvement. But in homogeneous settings and for certain model sizes, MPC adds 1–5% overhead without benefit.

3. **Benefits are model-dependent.** Mid-sized dense models with moderate per-token cost (4–6ms TPOT) benefit most from MPC. Very fast models outpace MPC's control loop; very large models exhibit dynamics that MPC's queue model does not capture.
4. **Simulation overestimates.** Our simulation predicted 23% latency improvement; cluster results show improvements ranging from −5% (overhead) to −49.8% depending on conditions, with many scenarios showing no benefit. Cluster evaluation is essential for routing research.

Broader lesson. The gap between simulation and reality in our study is itself a contribution. Routing algorithms that appear transformative in simulation may provide marginal or negative benefit in production. We advocate for cluster-scale evaluation as a standard requirement in LLM serving research, and for honest reporting of when proposed methods do *not* help alongside their successes.

Acknowledgments. We thank the anonymous reviewers for their feedback.

References

1. Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
2. Yu, G., et al.: Orca: A distributed serving system for transformer-based generative models. In: OSDI (2022)
3. Zhong, Y., et al.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
4. Patel, P., et al.: Splitwise: Efficient generative LLM inference using phase splitting. In: ISCA (2024)
5. Agrawal, A., et al.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: OSDI (2024)
6. vLLM Project: vLLM Router: A high-performance load balancer for large-scale serving. https://github.com/vllm-project/router (2025)
7. Mitzenmacher, M.: The power of two choices in randomized load balancing. IEEE TPDS 12(10), 1094–1104 (2001)
8. Camacho, E.F., Bordons, C.: Model Predictive Control. Springer (2013)
9. Williams, G., et al.: Model predictive path integral control using covariance variable importance sampling. arXiv:1707.02342 (2017)
10. Garcia, C.E., Prett, D.M., Morari, M.: Model predictive control: Theory and practice. Automatica 25(3), 335–348 (1989)
11. Low, S.H., Lapsley, D.E.: Optimization flow control—I: Basic algorithm and convergence. IEEE/ACM ToN 7(6), 861–874 (1999)
12. Lazic, N., et al.: Data center cooling using model-predictive control. In: NeurIPS (2018)
13. Zhang, Y., et al.: Model predictive control for cloud resource management. In: ICDCS (2021)
14. Karger, D., et al.: Consistent hashing and random trees. In: STOC (1997)
15. Eschenfeldt, P., Gamarnik, D.: Join the shortest queue with many servers. Mathematics of OR 43(3), 867–886 (2018)

16. Mao, H., et al.: Learning scheduling algorithms for data processing clusters. In: SIGCOMM (2019)
17. Dang, Y., et al.: AIOps: Real-world challenges and research innovations. In: ICSE-SEIP (2019)
18. Zhao, N., et al.: Identifying bad software changes via multimodal anomaly detection for online service systems. In: FSE (2021)
19. Cortez, E., et al.: Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: SOSP (2017)
20. Antsaklis, P.J., Michel, A.N.: Linear Systems. Birkhäuser (2006)